

NVISO Labs

Cyber security research, straight from the lab! 🐭



≡ Menu

Intercepting traffic from Android Flutter applications

Jeroen Beckers 📁 Mobile Security, Burp Suite, Android ⌚ August 13, 2019 ≡ 9 Minutes

Update: The explanation below explains the step for ARMv7. [For ARMv8 \(64bit\), see this blogpost.](#)

⚠️ Update August 2022 ⚠️

An update to this blog post was written and can be found [here](#). It covers both iOS and Android and a convenient script / Frida codeshare to use.

Flutter is Google's new open source mobile development framework that allows developers to write a single code base and build for Android, iOS, web and desktop. Flutter applications are written in **Dart**, a language created by Google more than 7 years ago.



It's often necessary to intercept traffic between a mobile application and the backend (either for a security assessment or a bounty hunt), which is typically done by adding Burp as an intercepting proxy. Flutter applications are a little bit more difficult to proxy, but it's definitely possible.

TL;DR

- Flutter uses Dart, which doesn't use the system CA store
- Dart uses a list of CA's that's compiled into the application
- Dart is not proxy aware on Android, so use ProxyDroid with iptables
- Hook the [session_verify_cert_chain](#) function in x509.cc to disable chain validation
- You might be able to use the script at the bottom of this article directly, or you can follow the steps below to get the right bytes or offset.

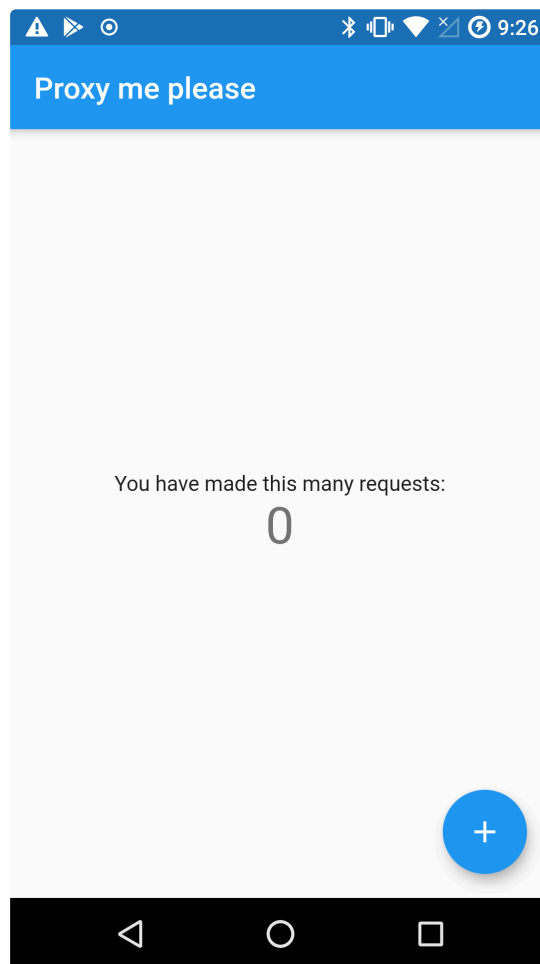
Test setup

In order to perform my tests, I [installed the flutter plugin](#) and created a Flutter application that comes with a default interactive button that increments a counter. I modified it to fetch a URL through the HttpClient class:

```
1  class _MyHomePageState extends State<MyHomePage> {  
2    int _counter = 0;  
3    HttpClient client;  
4  
5    _MyHomePageState()  
6    {  
7      _start();  
8    }  
9    void _start() async  
10   {
```

```
11     client = HttpClient();
12   }
13   void _incrementCounter() {
14     setState(() {
15       if(client != null)
16       {
17         client
18           .getUrl(Uri.parse('http://www.nviso.eu'))
19           .then((request) => request.close()) // se
20           .then((response) => print("SUCCESS - " +
21             _counter++);
22       }
23     });
24   }
```

The app can be compiled using `flutter build aot` and pushed to the device through `adb install`.



Every time we press the button, a call is sent to <http://www.nviso.eu> and if it's successful it is printed to the device logs.

On my device I have Frida installed through [Magisk-Frida-Server](#) and my Burp certificate is added to the system CA store with the [MagiskTrustUserCerts](#) module.

Unfortunately, Burp does not see any traffic passing through, even though the app logs indicate that the request was successful.

Sending traffic to the proxy through ProxyDroid/iptables

The HttpClient has a [findProxy](#) method and its documentation is pretty clear on this: By default all traffic is sent directly to the target server, without taking any proxy settings into account:

Sets the function used to resolve the proxy server to be used for opening a HTTP connection to the specified url . If this function is not set, direct connections will always be used.

– [findProxy documentation](#)

The application can set this property to `HttpClient.findProxyFromEnvironment` which searches for specific environment variables such as `http_proxy` and `https_proxy` . Even if the application would be compiled with this implementation, it would be pretty useless on Android since all applications are children of the initial zygote process which does not have these environment variables.

It's also possible to define a custom `findProxy` implementation that returns the preferred proxy. A quick modification on my test application indeed shows that this configuration sends all HTTP data to my proxy:

```
1 | client.findProxy = (uri) {  
2 |     return "PROXY 10.153.103.222:8888";  
3 | };
```

The screenshot displays the Burp Suite application interface. At the top, there are several tabs for different tools: Extender, Project options, User options, OpenAPI Parser, Add Custom Header, Dashboard, Target, Proxy (selected), Intruder, Repeater, Sequencer, Decoder, and Comparer. Below these, there are more tabs: Intercept, HTTP history (selected), WebSockets history, and Options. A filter bar indicates: "Filter: Hiding CSS, image and general binary content; matching expression nviso.eu; showing only high". Below the filter, a table lists intercepted requests:

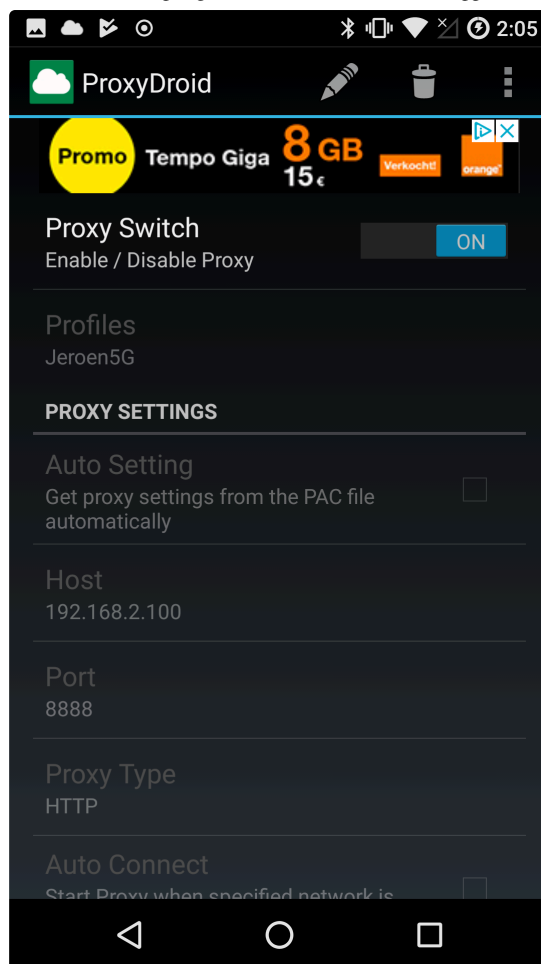
#	Host	Method	URL	Params	Edited
2737	http://www.nviso.eu	GET	/		

Below the table, there are tabs for "Request" and "Response". Under the "Request" tab, there are sub-tabs for "Raw" (selected), "Headers", and "Hex". The "Raw" view shows the following text:

```
GET / HTTP/1.1
user-agent: Dart/2.4 (dart:io)
Accept-Encoding: gzip, deflate
content-length: 0
host: www.nviso.eu
Connection: close
```

At the bottom, there is a search bar with a question mark icon, navigation buttons (<, +, >), a text input field containing "Type a search term", and a "0 matches" indicator.

Of course, we can't modify the application during a black-box assessment, so another approach is needed. Luckily, we always have the iptables fallback to route all traffic from the device to our proxy. On a rooted device, ProxyDroid handles this pretty well and we can see all HTTP traffic flowing through Burp.



ProxyDroid with root access using iptables

Intercepting HTTPS traffic

This is where it gets more tricky. If I change the URL to HTTPS, Burp complains that the SSL handshake fails. This is weird since my device is set up to include my Burp certificate as a trusted root CA.

After some research, I ended up on a [GitHub issue](#) that explains the issue for Windows, but the same is applicable to Android: Dart generates and [compiles its own Keystore](#) using [Mozilla's NSS library](#).

This means that we can't bypass SSL validation by adding our proxy CA to the system CA store. To solve this we have to dig into libflutter.so and figure out what we need to patch or hook in order to validate our certificate. Dart uses Google's BoringSSL to handle everything SSL related, and luckily both Dart and BoringSSL are open source.

When sending HTTPS traffic to Burp, the Flutter application actually throws an error, which we can take as a starting point:

E/flutter (10371): [ERROR:flutter/runtime/dart_isolate.cc(805)]

Unhandled exception:

E/flutter (10371): HandshakeException: Handshake error in client (OS

Error:

E/flutter (10371): NO_START_LINE(pem_lib.c:631)

E/flutter (10371): PEM routines(by_file.c:146)

E/flutter (10371): NO_START_LINE(pem_lib.c:631)

E/flutter (10371): PEM routines(by_file.c:146)

E/flutter (10371): CERTIFICATE_VERIFY_FAILED: self signed
certificate in certificate chain(handshake.cc:352))

E/flutter (10371): #0 _rootHandleUncaughtError.
(dart:async/zone.dart:1112:29)

E/flutter (10371): #1 _microtaskLoop
(dart:async/schedule_microtask.dart:41:21)

E/flutter (10371): #2 _startMicrotaskLoop
(dart:async/schedule_microtask.dart:50:5)

E/flutter (10371): #3 _runPendingImmediateCallback (dart:isolate-
patch/isolate_patch.dart:116:13)

E/flutter (10371): #4 _RawReceivePortImpl._handleMessage
(dart:isolate-patch/isolate_patch.dart:173:5)

The first thing we need to do is find this error in the [BoringSSL library](#). The error actually shows us where the error is triggered: handshake.cc:352 . [Handshake.cc](#) is indeed part of the BoringSSL library and does contain logic to perform certificate validation. The code at line 352 is shown below, and this is most likely the error we are seeing. The line numbers don't match exactly, but this is most likely the result of a version difference.

```
352 | if (ret == ssl_verify_invalid) {  
353 |     OPENSSL_PUT_ERROR(SSL, SSL_R_CERTIFICATE_VERIFY_FA  
354 |     ssl_send_alert(ssl, SSL3_AL_FATAL, alert);  
355 | }
```

This is part of the `ssl_verify_peer_cert` function which returns the `ssl_verify_result_t` enum which is defined in [ssl.h](#) at line 2290:

```
2290 | enum ssl_verify_result_t BORINGSSL_ENUM_INT {  
2291 |     ssl_verify_ok,  
2292 |     ssl_verify_invalid,  
2293 |     ssl_verify_retry,  
2294 | };
```

If we can change the return value of `ssl_verify_peer_cert` to `ssl_verify_ok` (`=0`), we should be good to go. However, a lot of stuff is going on in this method, and Frida can only (easily) change the return value of a function. If we change this value, it would still fail due to the `ssl_send_alert()` function call above (trust me, I tried 😊).

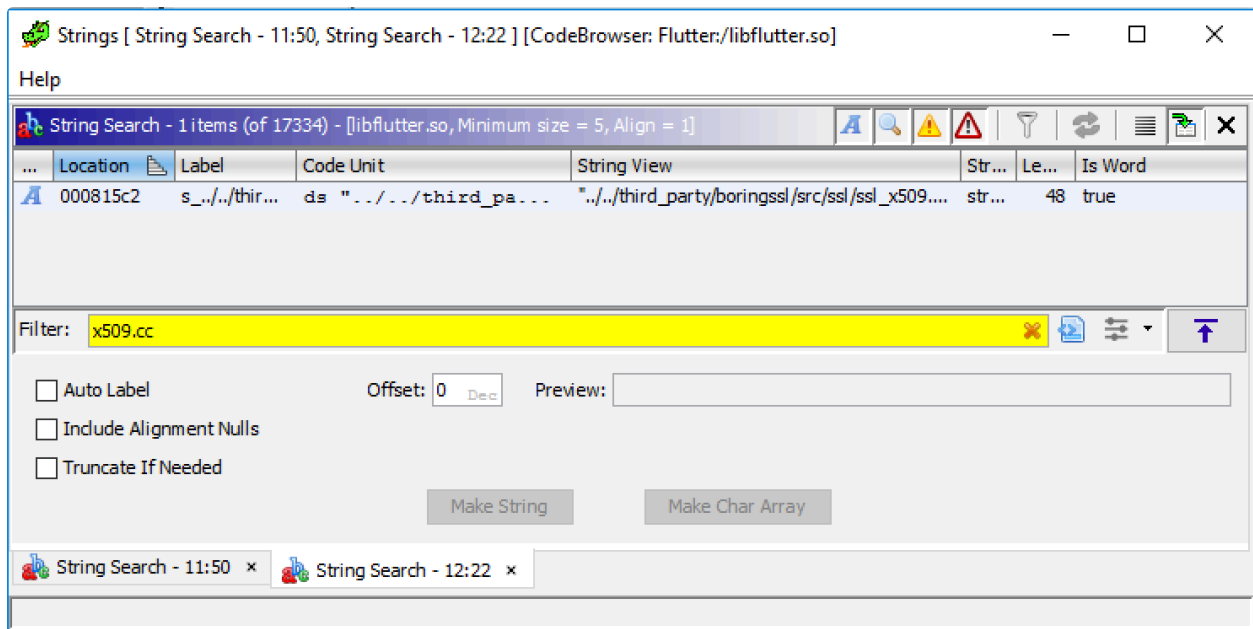
Let's find a better method to hook. Right above the snippet from `handshake.cc` is the following code, which is the actual part of the method that is validating the chain:

```
347 | ret = ssl->ctx->x509_method->session_verify_cert_chain  
348 |       hs->new_session.get(), hs, &alert)  
349 |       ? ssl_verify_ok  
350 |       : ssl_verify_invalid;
```

The `session_verify_cert_chain` function is defined in [ssl_x509.cc](#) at line 362. This function also returns a primitive datatype (boolean) and is a better candidate to hook. If a check fails in this function, it only reports the issue via `OPENSSL_PUT_ERROR`, but it doesn't have side effects like the `ssl_verify_peer_cert` function. The `OPENSSL_PUT_ERROR` is a macro defined in [err.h](#) at line 418 that includes the source filename. This is the same macro that was used for the error that made it to the Flutter app.

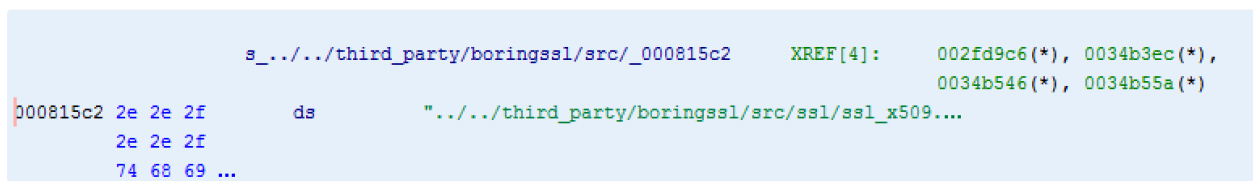
```
418 | #define OPENSSL_PUT_ERROR(library, reason) \  
419 |     ERR_put_error(ERR_LIB_##library, 0, reason, __FILE__
```

Now that we know which function we want to hook, we need to find it in `libflutter.so`. The `OPENSSL_PUT_ERROR` macro is called a few times in the `session_verify_cert_chain` function, which makes it easy to find the correct method using Ghidra. So import the library into Ghidra, use Search -> Find Strings and search for `x509.cc`.



Searching for the x509.cc string

There are only 4 XREFs so it's easy to go over them and find one that looks like the session_verify_cert_chain function:



Only 4 xrefs

One of the functions takes 2 ints, 1 'undefined' and contains a single call to OPENSSL_PUT_ERROR (FUN_00316500). In my version of libflutter.so, this is FUN_0034b330 . What you typically do now is calculate the offset of this function from one of the exported functions and hook it. I usually take a lazy approach where I copy the first 10 or so bytes of the function and check how often that pattern occurs. If it only occurs once, I know I found the function and I can hook it. This is useful because I can often use the same script for different versions of the library. With an offset based approach, this is more difficult.

```

*****
*                               FUNCTION                               *
*****
undefined FUN_0034b330()
    assume LRset = 0x0
    assume TMode = 0x1
    undefined    r0:1    <RETURN>
    FUN_0034b330+1      XREF[0,1]: 005be220(*)
    FUN_0034b330
0034b330 2d e9 f0 4f    push    { r4, r5, r6, r7, r8, r9, r10, r11, lr }
0034b334 a3 b0          sub     sp,#0x8c
0034b336 82 46          mov     r10,r0
0034b338 50 20          mov     r0,#0x50
0034b33a 10 70          strb   r0,[r2,#0x0]
0034b33c da f8 98 70    ldr.w   r7,[r10,#0x98]
0034b340 00 2f          cmp     r7,#0x0
0034b342 4c d0          beq     LAB_0034b3de
0034b344 38 68          ldr     r0,[r7,#0x0]
0034b346 00 28          cmp     r0,#0x0

```

So now we let Frida search the libflutter.so library for this pattern:

```

1 | var m = Process.findModuleByName("libflutter.so");
2 | var pattern = "2d e9 f0 4f a3 b0 82 46 50 20 10 70"
3 | var res = Memory.scan(m.base, m.size, pattern, {
4 |     onMatch: function(address, size){
5 |         console.log('[+] ssl_verify_result found at: ' +
6 |             },
7 |     onError: function(reason){
8 |         console.log('[!] There was an error scanning memo
9 |             },
10 |     onComplete: function()
11 |     {
12 |         console.log("All done")
13 |     }
14 | });

```

Running this script on my Flutter application gives just a single result:

```
(env) ~/D/Temp » frida -U -f be.nviso.flutter_app -l frida.js --no-pause
```

```
[LGE Nexus 5::be.nviso.flutter_app]-> [+] ssl_verify_result found at:
```

```
0x9a7f7040
```

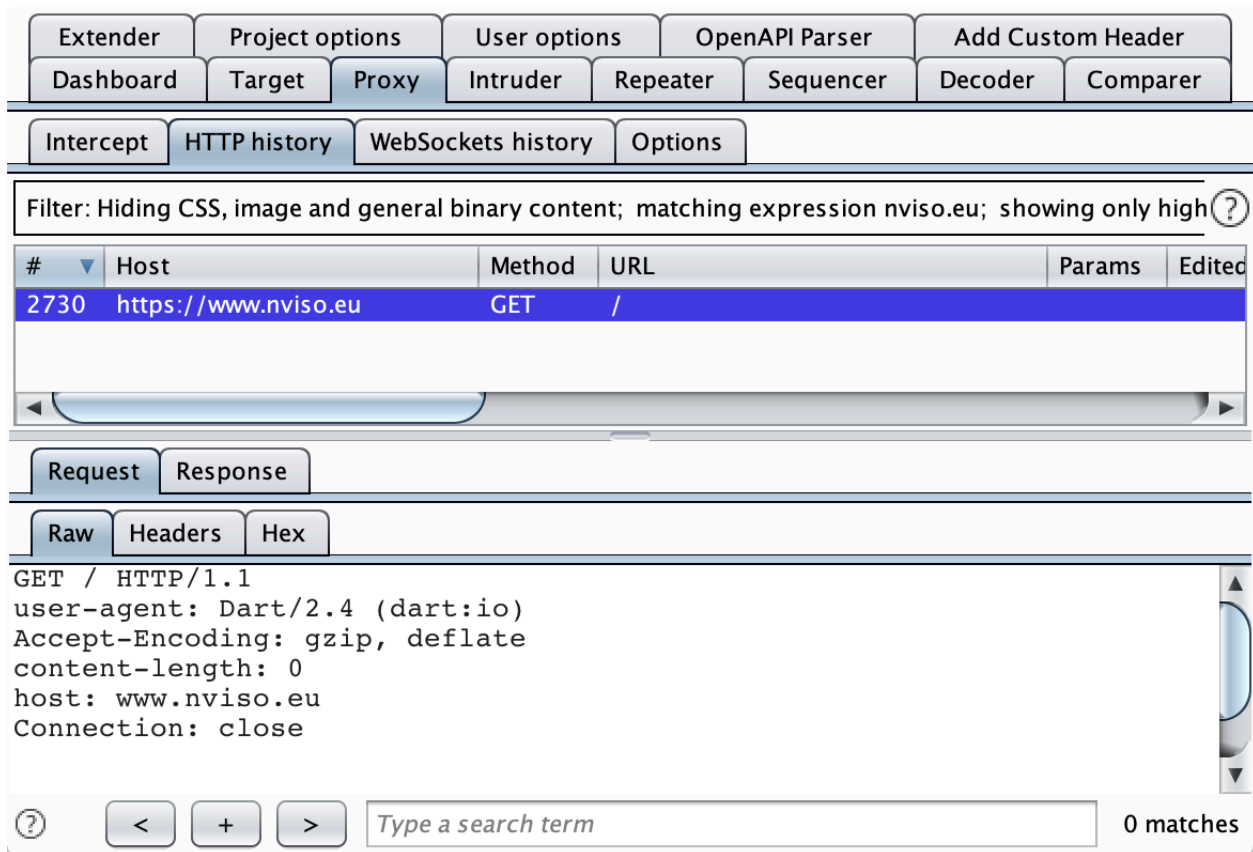
```
All done
```

Now we just need to use the Interceptor to change the return value to 1 (true):

```
1 | function hook_ssl_verify_result(address)
```

```
2  {
3    Interceptor.attach(address, {
4      onEnter: function(args) {
5        console.log("Disabling SSL validation")
6      },
7      onLeave: function(retval)
8      {
9        console.log("Retval: " + retval)
10       retval.replace(0x1);
11     }
12   });
13 }
14
15 function disablePinning()
16 {
17   var m = Process.findModuleByName("libflutter.so");
18   var pattern = "2d e9 f0 4f a3 b0 82 46 50 20 10 70"
19   var res = Memory.scan(m.base, m.size, pattern, {
20     onMatch: function(address, size){
21       console.log('[+] ssl_verify_result found at: ' +
22
23         // Add 0x01 because it's a THUMB function
24         // Otherwise, we would get 'Error: unable to inte
25       hook_ssl_verify_result(address.add(0x01));
26
27     },
28     onError: function(reason){
29       console.log('[!] There was an error scanning memo
30     },
31     onComplete: function()
32     {
33       console.log("All done")
34     }
35   });
36 }
37 setTimeout(disablePinning, 1000)
```

After setting up ProxyDroid and launching the application with this script, we can now finally see HTTPs traffic:



I've tested this on a few Flutter apps and this approach worked on all of them. As the BoringSSL library will most likely stay rather stable, this approach might work for some time to come.

Disable SSL Pinning (SecurityContext)

Finally, let's see how we can get around SSL Pinning. One way of doing this is by defining a new SecurityContext that contains specific certificates. While this is not technically SSL pinning (you don't protect against a compromised private key), it's often implemented to prevent against easy eavesdropping of the communication channel.

For my app, I added the following code to have it accept only my burp certificate. The [SecurityContext constructor](#) takes one argument, withTrustedRoots, which defaults to false.

```
1 |   ByteData data = await rootBundle.load('certs/burp.crt');
2 |   SecurityContext context = new SecurityContext();
3 |   context.setTrustedCertificatesBytes(data.buffer.asU
4 |   client = HttpClient(context: context);
```

The application will now automatically accept our Burp proxy as the certificate for any website, which shows that this method can be used to specify a specific

certificate that the application must comply to. If we now switch this to the nviso.eu certificate, we can no longer intercept the connection.

Fortunately, the Frida script listed above already bypasses this kind of root-ca-pinning implementation, as the underlying logic still depends on the same methods of the BoringSSL library.

Disable SSL Pinning (ssl_pinning_plugin)

One of the ways Flutter developers might want to perform ssl pinning is through the [ssl_pinning_plugin](#) flutter plugin. This plugin is actually designed to send one HTTPS connection and verify the certificate, after which the developer will trust the channel and perform non-pinned HTTPS requests:

With correct timing of ProxyDroid, this can already be circumvented, but let's just disable it anyway.

```
1 void testPin() async
2 {
3   List<String> hashes = new List<String>();
4   hashes.add("randomhash");
5   try
6   {
7     await SslPinningPlugin.check(serverURL: "https://
8
9     doImportanStuff()
10  } catch(e)
11  {
12    abortWithError(e);
13  }
14 }
```

The plugin is a bridge to a [Java implementation](#) which we can easily hook with Frida:

```
1 function disablePinning()
2 {
3   var SslPinningPlugin = Java.use("com.macif.plugin.s
4   SslPinningPlugin.checkConnexion.implementation = fu
5   {
6     console.log("Disabled SslPinningPlugin");
7     return true;
8   }
9 }
10
11 Java.perform(disablePinning)
```

Conclusion

This was a pretty fun ride, and it went quite smoothly since both Dart and BoringSSL are open source. Due to just a few interesting strings, it's pretty easy to find the correct place to disable the ssl verification logic, even without any symbols. My approach with scanning for the function prologue might not always work, but since BoringSSL is pretty stable, it should work for some time to come.

About the author

Jeroen Beckers is a mobile security expert working in the NVISO Cyber Resilience team and co-author of the OWASP Mobile Security Testing Guide (MSTG). He also loves to program, both on high and low level stuff, and deep diving into the Android internals doesn't scare him. You can find Jeroen on [LinkedIn](#).



Tagged: Mobile, android

Published by Jeroen Beckers

[View all posts by Jeroen Beckers](#)

◀ [Solving Flaggy Bird \(Google CTF 2019\)](#)

41 thoughts on “Intercepting traffic from Android Flutter applications”

Pingback: [Intercepting traffic from Android Flutter applications - ZRaven Consulting](#)

NULL

August 18, 2019 at 11:06 pm

Dear Jeroen,

Thanks a lot, this post is so awesome. I have one question though. Have you tried same thing for iOS?

Loading...

[↩ Reply](#)

N0n4me404

August 21, 2019 at 4:09 pm

Thank for your sharing. I was deal a flutter app and found another way to bypass. My method is I hook into the “connect” function in c and return ip resolved to my ip burp and use invisible proxy with root CA to bypass SSL pining.

Loading...

[↩ Reply](#)

Lukasz Pilorz

September 1, 2019 at 4:35 pm

Thanks for the post – I used a simpler method and it worked for all the Flutter Android apps I tested (just 4 of them, didn't check more): <https://hackmd.io/-kYFBsKiQleqrlyXR1eQFA?view>

The Android system CA store was sufficient, both for http.dart and HttpClient from dart:io. Tested on Pixel XL, Android 9 (LineageOS 16).

Loading...

[↩ Reply](#)**Champ**

May 21, 2022 at 2:12 am

burp throws "Client failed to negotiate TLS". Since burp presents its own per-host cert to the device and even if it is in system store, it won't work. Flutter has its own compiled CAs.

Loading...

[↩ Reply](#)

Pingback: [Burp](#) [Flutter App](#) – [NEWS.ALL](#)

Pingback: [Flutter Uygulamalarına Güvenlik Testi Nasıl Yapılır? – Big Data](#)

neeraj nigam

January 15, 2020 at 9:11 pm

Really beautiful article. learned Ghidra while following this tutorial. I found my offset for verifying function at "2d e9 f0 4f a3 b0 81 46 50 20 10 70" quite

similar to what mentioned in this post. Thanks a lot.

Loading...

[↩ Reply](#)

Jhon Mesa

April 27, 2020 at 12:13 pm

This is good stuff. I also achived the same thing by adding a few lines of code in my flutter app, as follow:

“`

```
import 'dart:io';
```

```
class MyHttpOverrides extends HttpOverrides {  
  @override  
  HttpClient createHttpClient(SecurityContext securityContext){  
    return super.createHttpClient(securityContext)  
      ..badCertificateCallback =  
        (X509Certificate cert, String host, int port) => true;  
  }  
}
```

```
@override  
String findProxyFromEnvironment(____) {  
  return 'PROXY 192.168.0.12:8080;'; // IP address of Burp proxy  
}  
}
```

```
void main() {  
  
  HttpOverrides.global = new MyHttpOverrides();  
  runApp(MyApp());  
}
```

“`

Loading...

[↩ Reply](#)**Jeroen Beckers**

April 29, 2020 at 9:11 pm

Hi John! If you have the source code and the ability to recompile the app, then adding your custom proxy in the findProxyEnvironment method is definitely the way to go. If you can't/don't, you'll have to use the setup listed above 😊 (Don't forget to remove that certificate check bypass from your app before going to prod 😊)

Loading...

[↩ Reply](#)**sparrowmobile2604@**

May 6, 2020 at 1:52 pm

Hi Jeroen,
Any app which can be used to intercept request for ios app developed on flutter like we have ProxyDroid for android.

Loading...

[↩ Reply](#)**Jeroen Beckers**

May 6, 2020 at 5:23 pm

Hi Sparrow, I have a post coming up on iOS. Unfortunately it's a bit more complicated. You have to set up an openvpn server and once

again disable the correct SSL validation logic using Frida.

Loading...

↩ Reply

Udin

May 10, 2020 at 9:49 am

Hi Jeroen,

First of all, thanks you for sharing, I learn a lot while trying to figure out ghidra & frida.

Actually this is the first time for me in intercepting traffic on Android Apps. I've follow all the walkthrough in this post, but I failed in placing hooks on flutter apps. I am sure the apps is flutter based android apps, but when list installed module (using Process.getModules function) there is no module with name "libflutter.so", do you why there is no libflutter.so module?

*note: when I extract the apk (apktool d apps-name.apk) I am sure there is libflutter.so file

Loading...

↩ Reply

Jeroen Beckers

June 8, 2020 at 9:02 am

Hi Udin, It is possible that you're intercepting too early, as the Application actually has to load the flutter library before you can hook it. You can increase the timeout of the setTimeout, maybe that helps.

Loading...

[↩ Reply](#)**Niraj Kharel**

June 2, 2021 at 7:13 am

Hello Udin, did you solved this issue?

Loading...

[↩ Reply](#)**Niraj Kharel**

June 2, 2021 at 8:21 am

I am using Genymotion Emulator for bypassing SSL Pinning in Flutter Application. Since the emulator is x86 architecture, I have installed ARM translation tool for running flutter application (ARMv7). The application runs smoothly, libflutter.so is loaded but the script is not able to find the module name with libflutter.so.

Loading...

Pingback: [Intercepting Flutter traffic on Android x64 – NVISO Labs](#)

Pingback: [Intercepting Flutter traffic on iOS – NVISO Labs](#)

Pingback: [Кроссплатформенные фреймворки для разработки приложений для Android и iOS - Stingray](#)

RabinDra

October 14, 2020 at 6:39 am

Hi Jeroen, an awesome article.

I have a flutter app but checking the resource there is no libflutter.so.

I used Process.enumerateModules but did not find any sign of libflutter.so.

The only place where I found libflutter.so was in

/data/app/app.name/lib/arm/libflutter.so along with libapp.so.

Loading...

↩ Reply

Flutter question?

May 11, 2022 at 9:50 pm

Did you solve the issue?

Loading...

↩ Reply

Jeroen Beckers

May 12, 2022 at 2:48 pm

Which problem are you trying to solve? You can find libflutter / libapp by using apktool on your apk, or you can extract them from the path that RabinDra posted after installing the apk to a device.

Loading...

Pingback: [Sniffing Android x86 HTTPs traffic with BurpSuite](https://blog.nviso.eu/2019/08/13/intercepting-traffic-from-android-flutter-applications/) | Tomas' Laboratory

Pingback: [Flutter based Mac OSX Thick Client SSL Pinning Bypass - NotSoSecure](#)

Pingback: [Intercept Android SSL Pinning on Flutter Based Android Application – Qlkwej](#)

Pingback: [\[Android\] Cara membypass SSL pin pada aplikasi berbasis flutter | by Rian Ady | Feb, 2021 • New Of Games](#)

Pingback: [Reversing a Flutter app by recompiling Flutter Engine – Tinyhack.com](#)

Pingback: [Tráfico HTTPS de app Flutter con Burp usando Genymotion PaaS - DontTouchMyNet](#)

micro

April 5, 2021 at 11:56 pm

hi.

i increased setTimeout too, but i have same problem and it did not find any sign of libflutter.so.

please help

I am sure there is libflutter.so file

Loading...

[↩ Reply](#)

Jeroen Beckers

April 6, 2021 at 10:30 am

If you decompile the app with apktool, can you find a libflutter.so?
Some apps dont' load the flutter library at start time, so you could

hook the `System.loadLibrary` call and wait for the library to finish loading before hooking the SSL logic.

Loading...

↩ Reply

Tom

November 12, 2021 at 9:59 pm

latest libflutter on x64 works for me with pattern
`var pattern = "ff 03 05 d1 fd 7b 0f a9 fa 67 10 a9";`
and because of x64 normal address
`hook_ssl_verify_result(address);`

Loading...

↩ Reply

Pingback: [MOV AX, BX Code depilation salon: Articles, Code samples, Processor code documentation, Low-level programming, Working with debuggers Bypassing SSL pinning on Android Flutter Apps with Ghidra](#)

Pingback: [Подборка материалов по мобильной безопасности «Awesome Mobile Security» | vkwebsite.site](#)

Pingback: [Bypassing SSL pinning on Android Flutter Apps with Ghidra | Flagrant Malfeasance](#)

Pingback: [Intercept Flutter traffic on iOS and Android \(HTTP/HTTPS/Dio Pinning\) – NVISO Labs](#)

Pingback: [Proxying Android app traffic – Common issues / checklist – NVISO Labs](#)

Darsh

April 3, 2024 at 5:10 pm

Hi Jeroen,

In my case lib folder contains “armeabi-v7a” and “arm64-v8a” both architecture folders, so might application supports both architectures.

However, when i open libflutter.so in Ghidra i searched for “x509.cc” and found 4 unique functions s shown below:

s_../../third_party/boringssl/src/_002ed103 XREF[5]:

FUN_00883774:0088380f(*),

FUN_00883f7a:008841c9(*),

FUN_008842e4:008844a8(*),

FUN_008845c1:008846a7(*),

FUN_008845c1:008846c2(*)

002ed103 2e 2e 2f ds “../../third_party/boringssl/src/ssl/ssl_x509.

2e 2e 2f

74 68 69

how to identify that which function fulfills below requirement:

“One of the functions takes 2 ints, 1 ‘undefined’ and contains a single call to OPENSSL_PUT_ERROR (FUN_00316500). In my version of libflutter.so, this is FUN_0034b330”

waiting for your response

Loading...

[↩ Reply](#)

Bo

April 6, 2024 at 6:37 pm

Hi, Jeroen

Im trying to replicate this using that app on your GitHub. However, the interceptor.process does not execute. May I know what version of frida-server (the one pushed in the mobile device) and frida (the one in the computer) are you using for this. Thanks

Loading...

↪ Reply

Jeroen Beckers

April 6, 2024 at 11:33 pm

Hi Bo! I recently did a rewrite of the script; Can you try out the latest version on GitHub and open a Github issue in case it doesn't work?

Loading...

↪ Reply

Bo

April 7, 2024 at 7:56 am

Hi Jeroen,

Thank you for replying. I opened a github issue. I made it as detailed as possible.

Can you please take a look at it.

Thank you.

Loading...

Bo

April 16, 2024 at 1:09 pm

Hi, Jeroen,

It's me again. I am getting this error that is showing on the app itself
"connection closed before full header was received".

This only happens when Proxydroid is open but when I close it the app works perfectly

I can also intercept the other functionality of the app but other functions like adding an account, it does not work. I am getting that "connection closed" error.

Any idea on what is this?..

Thanks

Loading...

[↩ Reply](#)

Leave a Reply



[NVISO Homepage](#)

[Jobs](#)

Info and support

info@nviso.eu

Got hacked?

csirt@nviso.eu